arXiv:2502.00873v1 [cs.AI] 2 Feb 2025

Language Models Use Trigonometry to Do Addition

Subhash Kantamneni [1] **Max Tegmark** [1]

Abstract

Mathematical reasoning is an increasingly important indicator of large language model (LLM) capabilities, yet we lack understanding of how LLMs process even simple mathematical tasks. To address this, we reverse engineer how three mid-sized LLMs compute addition. We first discover that numbers are represented in these LLMs as a generalized helix, which is strongly causally implicated for the tasks of addition and subtraction, and is also causally relevant for integer division, multiplication, and modular arithmetic. We then propose that LLMs compute addition by manipulating this generalized helix using the "Clock" algorithm: to solve $a + b$, the helices for a and b are manipulated to produce the $a + b$ answer helix which is then read out to model logits. We model influential MLP outputs, attention head outputs, and even individual neuron preactivations with these helices and verify our understanding with causal interventions. By demonstrating that LLMs represent numbers on a helix and manipulate this helix to perform addition, we present the first representation-level explanation of an LLM's mathematical capability.

1. Introduction

Large language models (LLMs) display surprising and significant aptitude for mathematical reasoning (Ahn et al., 2024; Satpute et al., 2024), which is increasingly seen as a benchmark for LLM capabilities (OpenAI; Glazer et al., 2024). Despite LLMs' mathematical proficiency, we have limited understanding of how LLMs process even simple mathematical tasks like addition. Understanding mathematical reasoning is valuable for ensuring LLMs' reliability, interpretability, and alignment in high-stakes applications.

In this study, we reverse engineer how GPT-J, Pythia-6.9B, and Llama3.1-8B compute the addition problem $a + b$ for $a, b \in [0, 99]$. Remarkably, we find that LLMs use a form of the "Clock" algorithm to compute addition, which was previously proposed by Nanda et al. (2023a) as a mechanistic explanation of how one layer transformers compute modular addition (and later named by Zhong et al. (2023)).

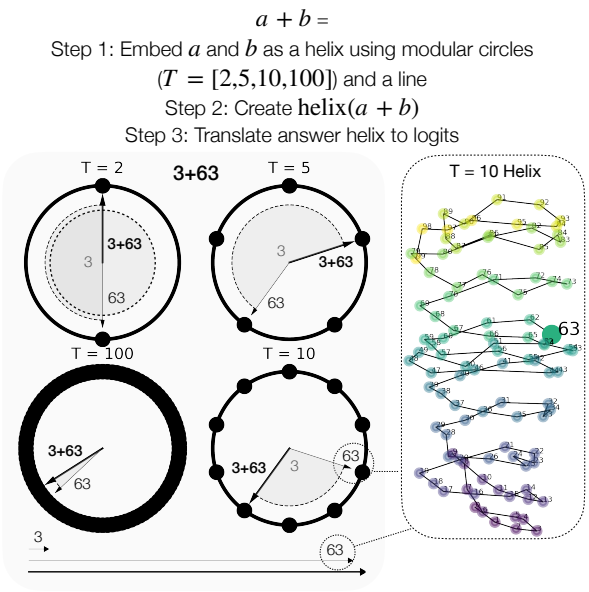


Figure 1. **Illustrating the Clock algorithm.** We find that LLMs represent numbers on a helix. When computing the addition problem $a + b$, LLMs rotate the a and b helices, as if on a clock, to create the $a + b$ helix and read out the final answer.

To compute $a + b$, all three LLMs represent a and b as a helix on their tokens and construct $\mathrm{helix}(a + b)$ on the last token, which we verify with causal interventions. We then focus on how GPT-J implements the Clock algorithm by investigating MLPs, attention heads, and even specific neurons. We find that these components can be understood as either constructing the $a + b$ helix by manipulating the a and b helices, or using the $a + b$ helix to produce the answer in the model's logits. We visualize this procedure in Fig. 1 as rotating the dial of a clock.

Our work is in the spirit of mechanistic interpretability (MI), which attempts to reverse engineer the functionality of machine learning models. However, most LLM MI research focuses either on identifying circuits, which are the minimal set of model components required for computations,

[1]Massachusetts Institute of Technology. Correspondence to: Subhash Kantamneni <subhashk@mit.edu>.

or understanding features, which are the representations of concepts in LLMs. A true mechanistic explanation requires understanding both how an LLM represents a feature and how downstream components manipulate that feature to complete a task. To our knowledge, we are the first work to present this type of description of an LLM's mathematical capability, identifying that LLMs represent numbers as helices and compute addition by manipulating these helices with the interpretable Clock algorithm.

2. Related Work

Circuits. Within mechanistic interpretability, circuits research attempts to understand the key model components (MLPs and attention heads) that are required for specific functionalities (Olah et al., 2020; Elhage et al., 2021). For example, Olsson et al. (2022) found that in-context learning is primarily driven by induction attention heads, and Wang et al. (2023) identified a sparse circuit of attention heads that GPT-2 uses to complete the indirect object of a sentence. Understanding how multilayer perceptrons (MLPs) affect model computation has been more challenging, with Nanda et al. (2023b) attempting to understand how MLPs are used in factual recall, and Hanna et al. (2023) investigating MLP outputs while studying the greater-than operation in GPT-2.

Features. Another branch of MI focuses on understanding how models represent human-interpretable concepts, known as features. Most notably, the Linear Representation Hypothesis posits that LLMs store features as linear directions (Park et al., 2023; Elhage et al., 2022), culminating in the introduction of sparse autoencoders (SAEs) that decompose model activations into sparse linear combinations of features (Huben et al., 2024; Bricken et al., 2023; Templeton et al., 2024; Gao et al., 2024; Rajamanoharan et al., 2024). However, recent work from Engels et al. (2024) found that some features are represented as non-linear manifolds, for example the days of the week lie on a circle. Levy & Geva (2024) and Zhu et al. (2025) model LLMs' representations of numbers as a circle in base 10 and as a line respectively, although with limited causal results. Recent work has bridged features and circuits research, with Marks et al. (2024) constructing circuits from SAE features and Makelov et al. (2024) using attention-based SAEs to identify the features used in Wang et al. (2023)'s IOI task.

Reverse engineering addition. Liu et al. (2022) first discovered that one layer transformers generalize on the task of modular addition when they learn circular representations of numbers. Following this, Nanda et al. (2023a) introduced the "Clock" algorithm as a description of the underlying angular addition mechanisms these transformers use to generalize. However, Zhong et al. (2023) found the "Pizza" algorithm as a rivaling explanation for some transformers, illustrating the complexity of decoding even small models. Stolfo et al. (2023) identifies the circuit used by LLMs in addition problems, and Nikankin et al. (2024) claims that LLMs use heuristics implemented by specific neurons rather than a definite algorithm to compute arithmetic. Zhou et al. (2024) analyze a fine-tuned GPT-2 and found that Fourier components in numerical representations are critical for addition, while providing preliminary results that larger base LLMs might use similar features.

3. Problem Setup

Models As in Nikankin et al. (2024), we analyze 3 LLMs: GPT-J (6B parameters) (Wang & Komatsuzaki, 2021), Pythia-6.9B (Biderman et al., 2023), and Llama3.1-8B (Grattafiori et al., 2024). All three models are autoregressive transformers which process tokens $x_0, ..., x_n$ to produce probability distributions over the likely next token x_{n+1} (Vaswani et al., 2017). The ith token is embedded as L hidden state vectors (also known as the residual stream), where L is the number of layers in the transformer. Each hidden state is the sum of multilayer perceptron (MLP) and attention (attn) layers.

$$\begin{aligned} h_i^l &= h_i^{l-1} + a_i^l + m_i^l, \\ a_i^l &= \mathrm{attn}^l\left(h_1^{l-1}, h_2^{l-1}, \ldots, h_i^{l-1}\right), \\ m_i^l &= \mathrm{MLP}^l(a_i^{(l)} + h_i^{l-1}). \end{aligned} \tag{1}$$

GPT-J and Pythia-6.9B use simple MLP implementations, namely $\mathrm{MLP}(x) = \sigma\left(xW_{\mathrm{up}}\right)W_{\mathrm{down}}$, where $\sigma(x)$ is the sigmoid function. Llama3.1-8B uses a gated MLP, $\mathrm{MLP}(x) = \sigma\left(xW_{\mathrm{gate}}\right) \circ \left(xW_{\mathrm{in}}\right)W_{\mathrm{out}}$, where $\circ$ represents the Hadamard product (Liu et al., 2021). GPT-J tokenizes the numbers $[0, 361]$ (with a space) as a single token, Pythia-6.9B tokenizes $[0, 557]$ as a single token, and Llama3.1-8B tokenizes $[0, 999]$ as a single token. We focus on the single-token regime for simplicity.

Data To ensure that answers require only a single token for all models, we construct problems $a + b$ for integers $a, b \in [0, 99]$. We evaluate all three models on these 10,000 addition problems, and find that all models can competently complete the task: GPT-J achieves 80.5% accuracy, Pythia-6.9B achieves 77.2% accuracy, and Llama3.1-8B achieves 98.0% accuracy. For the prompts used and each model's performance heatmap by a and b, see Appendix A. Despite Llama3.1-8B's impressive performance, in the main paper we focus our analysis on GPT-J because its simple MLP allows for easier neuron interpretation. We report similar results for Pythia-6.9B and Llama3.1-8B in the Appendix.

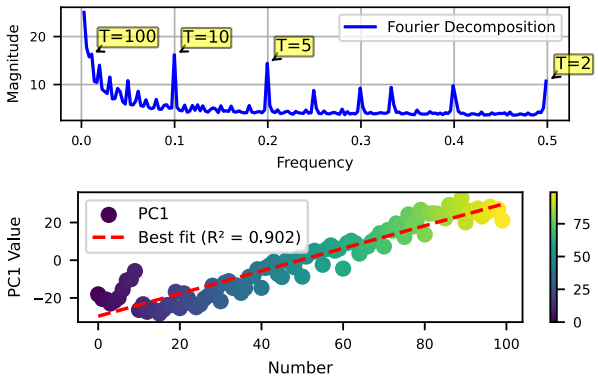


Figure 2. **Number representations are both periodic and linear.** *Top* The residual stream after layer 0 in GPT-J is sparse in the Fourier domain when batching the hidden states for $a \in [0, 360]$ together. We average the magnitude of the Fourier transform of the batched matrix h^0_{360} across the model dimension. *Bottom* In addition to this periodicity, the first PCA component is roughly linear in a for $a \in [0, 99]$.

4. LLMs Represent Numbers as a Helix

To generate a ground up understanding of how LLMs compute $a + b$, we first aim to understand how LLMs represent numbers. To identify representational trends, we run GPT-J on the single-token integers $a \in [0, 360]$. We do not use $a = 361$ because 360 has more integer divisors, allowing for a simpler analysis of periodic structure. We conduct analysis on h^0_{360}, which is the residual stream following layer 0 with shape $[360, \text{model_dim}]$. We choose to use the output of layer 0 rather than directly analyzing the embeddings because prior work has shown that processing in layer 0 is influential for numerical tasks (Nikankin et al., 2024).

4.1. Investigating Numerical Structure

Linear structure. To investigate structure in numerical representations, we perform a PCA (F.R.S., 1901) on h^0_{360} and find that the first principal component (PC1) for $a \in [0, 360]$ has a sharp discontinuity at $a = 100$ (Fig. 15, Appendix B), which implies that GPT-J uses a distinct representation for three-digit integers. Instead, in the bottom of Fig. 2, we plot PC1 for h^0_{99} and find that it is well approximated by a line in a. Additionally, when plotting the Euclidean distance between a and $a + \delta n$ for $a \in [0, 9]$ (Fig. 14, Appendix B), we see that the distance is locally linear. The existence of linear structure is unsurprising - numbers are semantically linear, and LLMs often represent concepts linearly.

Periodic Structure. We center and apply a Fourier transform to h^0_{360} with respect to the number a being represented and the model_dim. In Fig. 2, we average the resulting spectra across model_dim and observe a sparse Fourier domain with high-frequency components at $T = [2, 5, 10]$. Additionally, when we compare the residual streams of all pairs of integers a_1 and a_2, we see that there is distinct periodicity in both their Euclidean distance and cosine similarity (Fig. 13, Appendix B). These Fourier features were also identified by Zhou et al. (2024), and although initially surprising, are sensible. The units digit of numbers in base 10 is periodic ($T = 10$), and it is reasonable that qualities like evenness ($T = 2$) are useful for tasks.

4.2. Parameterizing the Structure as a Helix

To account for both the periodic and linear structure in numbers, we propose that numbers can be modeled helically. Namely, we posit that h^l_a, the residual stream immediately preceding layer l for some number a, can be modeled as

$$\begin{aligned} h^l_a &= \text{helix}(a) = CB(a)^T, \\ B(a) &= \left[a, \cos\left(\frac{2\pi}{T_1}a\right), \sin\left(\frac{2\pi}{T_1}a\right), \right. \\ &\quad \left. \ldots, \cos\left(\frac{2\pi}{T_k}a\right), \sin\left(\frac{2\pi}{T_k}a\right)\right]. \end{aligned} \tag{2}$$

C is a matrix applied to the basis of functions $B(a)$, where $B(a)$ uses k Fourier features with periods $T = [T_1, \ldots T_k]$. The $k = 1$ case represents a regular helix; for $k > 1$, the independent Fourier features share a single linear direction. We refer to this structure as a generalized helix, or simply a helix for brevity.

We identify four major Fourier features: $T = [2, 5, 10, 100]$. We use the periods $T = [2, 5, 10]$ because they have significant high frequency components in Fig. 2. We are cautious of low frequency Fourier components, and use $T = 100$ both because of its significant magnitude, and by applying the inductive bias that our number system is base 10.

4.3. Fitting a Helix

We fit our helical form to the residual streams on top of the a token for our $a + b$ dataset. In practice, we first use PCA to project the residual stream at each layer to 100 dimensions. To ensure we do not overfit with Fourier features, we consider all combinations of k Fourier features, with $k \in [1, 4]$. If we use k Fourier features, the helical fit uses $2k + 1$ basis functions (one linear component, $2k$ periodic components). We then use linear regression to find some coefficient matrix C_{PCA} of shape $100 \times 2k + 1$ that best satisfies $\text{PCA}(h^l_a) = C_{\text{PCA}}B(a)^T$. Finally, we use the inverse PCA transformation to project C_{PCA} back into the model's full residual stream dimensionality to find C.

We visualize the quality of our fit for layer 0 when using all $k = 4$ Fourier features with $T = [2, 5, 10, 100]$ in Fig. 3. To do so, we calculate $C^\dagger h$, where $C^\dagger$ is the Moore-Penrose pseudo-inverse of C. Thus, $C^\dagger h$ represents the